实验题一：阅读下面程序，对代码进行分析，说明代码的执行过程，说明 number_compare 与 number_move 是干什么的（代码分析内容写在解题思路中），将函数 bubble_sort 改为函数模板的形式，使之能支持其他类型的数据，示例代码如下。

代码：

```
#include <iostream>
using namespace std;
void bubble_sort(int *items, int count)
{
    int a, b;
    int number_compare = 0;
    int number_move = 0;
    bool indicator = 0;
    int t;
    for(a = 1; a < count; a++)
    {
        for(b = count - 1; b >= a; b--)
        {
            number_compare++;
            if(items[b - 1] > items[b])
            {
                number_move++;
                t = items[b - 1];
                items[b - 1] = items[b];
                items[b] = t;
                indicator = 1;
            }
        }
        if (indicator == 0)
            break;
    }

    cout<<"Number of comparisons: "<<number_compare<<endl;
    cout<<"Number of moves: "<<number_move<<endl;
}

int main()
{
    int arr[] = { 5, 4, 3, 2, 1};
    int len = (int) sizeof(arr) / sizeof(*arr);
    bubble_sort(arr, len);
    for (int i = 0; i < len; i++)
        cout << arr[i] << ' ';
    cout << endl;
```

```
    return 0;
}
```

bubble_sort 修改为函数模板后的主程序及运行结果：

主函数：

```
int main()
{
    int arr[] = { 5, 4, 3, 2, 1};
    int len = (int) sizeof(arr) / sizeof(*arr);
    bubble_sort(arr, len);
    for (int i = 0; i < len; i++)
        cout << arr[i] << ' ';
    cout << endl;
    float arrf[] = { 17.5, 19.1, 0.6, 1.9, 10.5};
    len = (float) sizeof(arrf) / sizeof(*arrf);
    bubble_sort(arrf, len);
    for (int i = 0; i < len; i++)
        cout << arrf[i] << ' ';
    cout << endl;
    char arrc[] = { 'g','f' ,'d' ,'s' ,'a' };
    len = (char) sizeof(arrc) / sizeof(*arrc);
    bubble_sort(arrc, len);
    for (int i = 0; i < len; i++)
        cout << arrc[i] << ' ';
    cout << endl;
    return 0;
}
```

运行结果：

```
Number of comparisons: 10
Number of moves: 10
1 2 3 4 5
Number of comparisons: 10
Number of moves: 6
0.6 1.9 10.5 17.5 19.1
Number of comparisons: 10
Number of moves: 7
a d f g s
```

实验题二：定义一个名为 Point 的类，用于表示空间中的二维点，该类包含三个私有成员变量，x 和 y 表示点的坐标，均为 double 类型，label 是一个字符指针，用于存储点的标签。现要求实现如下功能：

1）实现不带参数的构造函数，调用该函数时 x=y=0，label 为"Original Point"；
2）实现带三个参数的构造函数；
3）实现一个拷贝构造函数，并在解题思路中说明是深拷贝还是浅拷贝，为什么；
4）实现一个委托构造函数，委托 Point(0, 0, label_)函数实现对象的构造；
5）实现析构函数，释放空间；
6）实现一个函数 set_Point，用以改变点的坐标及标签；
7）实现一个函数 print，用以打印点的相关信息。

Point 的部分内容，主函数及输出示例如下：

```
class Point{
private:
    double x, y;
    char* label;
public:
    // 补充；
}
int main() {
    Point Original_Point;
    Original_Point.print();
    Point First_Point(1.0, 2.0, "First Point");
    First_Point.print();
    Point Second_Point(First_Point);
    Second_Point.print();
    cout<<"很显然，这两个点是一样的！请修改！"<<endl;
    Second_Point.set_Point(6.0, 8.0, "Second Point");
    Second_Point.print();
    Point Third_Point("Third Point");
    Third_Point.print();
    return 0;
}
```

```
Original Point调用了不带参数的构造函数！
Original Point的坐标为： (0, 0)
First Point调用了带三个参数的构造函数！
First Point的坐标为： (1, 2)
调用了拷贝构造函数！
First Point的坐标为： (1, 2)
很显然，这两个点是一样的！请修改！
Second Point的坐标为： (6, 8)
Third Point调用了带三个参数的构造函数！
Third Point委托了Point(0, 0, label_)函数实现对象的构造！
Third Point的坐标为： (0, 0)
Third Point调用了析构函数！
Second Point调用了析构函数！
First Point调用了析构函数！
Original Point调用了析构函数！
```

实验题三：设计一个学生类 Student，包含以下私有成员：char*类型的 ID，char*类型的 name，char*类型的 address，int 类型的 age。根据主函数及输出的内容，完成如下任务：

1）设计默认构造函数；
2）设计带参数的构造函数；
3）设计拷贝构造函数；
4）设计移动构造函数；
5）设计委托构造函数；
6）设计析构函数，释放空间；
7）设计信息打印函数，信息设置函数，姓名获取函数等。

主函数及输出示例如下：

```
int main() {
    Student Naruto;
    Naruto.Inf_Print();
    Student Mikoto_Misaka("2024020002", "御坂美琴", "学院都市", 16);
    cout<<Mikoto_Misaka.Get_name()<<"在内存中的地址是："<<&Mikoto_Misaka <<endl;
    Mikoto_Misaka.Inf_Print();
    Student Sasuke_Uchiha(Naruto);
    Sasuke_Uchiha.Set_ID("2024020003");
    Sasuke_Uchiha.Set_name("宇智波佐助");
    Sasuke_Uchiha.Inf_Print();
    Student Shirai_Kuroko("2024020004", "白井黑子");
    Shirai_Kuroko.Inf_Print();
    Student Sister_Misaka_I = std::move(Mikoto_Misaka);
    Sister_Misaka_I.Inf_Print();
    Sister_Misaka_I.Set_Inf("2024020005", "御坂妹", "学院都市", 1);
    Sister_Misaka_I.Inf_Print();
    return 0;
}
```

```
漩涡鸣人调用了默认的构造函数！
ID: 2024020001, 姓名: 漩涡鸣人, 年龄: 18, 来自: 木叶村
御坂美琴调用了带参数的构造函数！
御坂美琴在内存中的地址是: 0x61fdd0
ID: 2024020002, 姓名: 御坂美琴, 年龄: 16, 来自: 学院都市
漩涡鸣人调用了拷贝构造函数！
ID: 2024020003, 姓名: 宇智波佐助, 年龄: 18, 来自: 木叶村
白井黑子调用了带参数的构造函数！
白井黑子委托带参数的构造函数构造对象！
ID: 2024020004, 姓名: 白井黑子, 年龄: 16, 来自: 学院都市
御坂美琴调用了移动构造函数！
ID: 2024020002, 姓名: 御坂美琴, 年龄: 16, 来自: 学院都市
ID: 2024020005, 姓名: 御坂妹, 年龄: 1, 来自: 学院都市
御坂妹调用了析构函数！
白井黑子调用了析构函数！
宇智波佐助调用了析构函数！
空对象调用了析构函数！
漩涡鸣人调用了析构函数！
```

在解题思路中分析：Mikoto_Misaka 和 Sister_Misaka_I 是同一个对象吗？调用移动构造函数后 Mikoto_Misaka 这个对象是否还存在？